

Face Image Generation using Denoising Diffusion Probabilistic Models on CelebA-HQ

J.J. Paul, J.K. Jose, S. Syrus, K.M. Mathew, D. Vinod Kumar

EEE M068 Applied Machine Learning Group 30

University of Surrey, Guildford, United Kingdom

GitHub: <https://github.com/johanjoepaul/eee068-diffusion-face-generation>

Abstract—This project investigates image generation using Denoising Diffusion Probabilistic Models (DDPMs), with the main focus on unconditional human face generation using the CelebA-HQ dataset. A U-Net denoising model was trained to predict Gaussian noise added to images at randomly sampled diffusion timesteps. The implemented pipeline covers dataset preparation, preprocessing, forward diffusion visualisation, model construction, DDPM training, image generation, real-versus-generated comparison, and quantitative evaluation using Fréchet Inception Distance (FID). Several experiments were performed across image resolution, batch size, scheduler type, augmentation strategy, learning rate, and training duration. A preliminary diffusion pipeline was also explored using a butterfly image dataset before extending the work to the more challenging CelebA-HQ face generation task. The strongest final CelebA-HQ configuration used 128×128 images, batch size 8, a linear DDPM scheduler, and 150 training epochs, achieving an FID score of 94.41. The results show that extended training at 128×128 resolution performed better than higher-resolution 256×256 experiments under the available compute budget. The final model generates recognisable face-like images, although artefacts such as colour shifts, blurred details, and imperfect facial structure remain visible.

I. INTRODUCTION

Generative modelling is an important area of machine learning and computer vision because it allows models to learn complex image distributions and create new synthetic samples. Earlier methods such as Generative Adversarial Networks (GANs) showed strong image generation ability, but they can be difficult to train due to instability, mode collapse, and sensitivity to hyperparameters. Diffusion models provide an alternative by learning image generation as a gradual denoising process. Instead of creating an image in a single forward pass, a diffusion model begins from random Gaussian noise and iteratively removes noise until a realistic image is formed.

This project focuses mainly on unconditional face generation using the CelebA-HQ dataset. Human faces are a suitable target for this task because they contain both regular structure and high visual variation. A good model must learn general face layout, colour distribution, lighting, hair regions, pose variation, and local facial features. At the same time, face generation is challenging because small distortions in eyes, mouth, skin tone, or symmetry are easy to notice.

The project began with a simpler butterfly image generation pipeline to understand the overall diffusion workflow. This helped validate the core stages of dataset loading, forward noising, reverse denoising, training, and sample generation.

The pipeline was then extended to CelebA-HQ, where the task became more complex due to the higher importance of structural realism.

The main objectives of the project were to build an end-to-end DDPM pipeline, train a U-Net denoising model from scratch, generate new face images, compare generated images against real held-out samples, and evaluate the results using FID. The main contributions are: (i) a complete DDPM implementation for CelebA-HQ face generation; (ii) reproducible preprocessing and train-test splitting; (iii) multiple experimental comparisons across resolution, scheduler, augmentation, and epoch count; and (iv) qualitative and quantitative analysis of the final generated images.

II. LITERATURE REVIEW

Denoising Diffusion Probabilistic Models were introduced by Ho et al. as a generative modelling framework based on reversing a gradual Gaussian noising process [1]. During the forward process, clean images are progressively corrupted over many timesteps. During training, a neural network learns to predict the noise added at a sampled timestep. During generation, the model starts from random noise and repeatedly denoises it until an image is produced. This makes DDPMs attractive because the training objective is simple and stable, commonly using mean squared error between the predicted and true noise.

Score-based generative modelling provides a related interpretation of diffusion models by learning gradients of the data distribution and sampling through stochastic differential equations [2]. This broader view connects diffusion models to continuous-time generative modelling. Later research showed that diffusion models can outperform GANs on image synthesis when sufficiently scaled and carefully tuned [3]. Improved DDPMs also introduced refinements such as better noise schedules and variance learning, improving sampling quality and efficiency [4].

GAN-based models remain important historical baselines for image synthesis. Progressive GANs demonstrated that gradually increasing image resolution can improve high-resolution image generation and training stability [5]. However, GAN training can be unstable because the generator and discriminator are optimised against each other. Diffusion models avoid this adversarial setup and instead optimise a denoising objective, which is easier to monitor during training.

The U-Net architecture is widely used in diffusion models because it combines an encoder-decoder structure with skip connections [6]. This allows the network to preserve fine image details while also learning high-level spatial context. In this project, the U-Net receives a noisy RGB image and timestep information, then predicts the noise residual. AdamW was used as the optimiser because decoupled weight decay improves regularisation compared with traditional Adam weight decay [7]. The implementation used the HuggingFace Diffusers library, which provides standard DDPM schedulers, U-Net components, and generation pipelines [8].

Generated image quality was evaluated using Fréchet Inception Distance, which compares the feature distributions of real and generated images using an Inception network [9]. Lower FID indicates that generated samples are closer to the real image distribution. However, FID can fluctuate with sample count and random seed, so visual inspection was also used. Future extensions could use DDIM sampling to reduce the number of reverse diffusion steps [10], or latent diffusion models to move the diffusion process into a compressed representation for more efficient high-resolution generation [11].

III. METHODOLOGY

A. Dataset

The main dataset used in this project was CelebA-HQ at 256×256 resolution. Although the full dataset contains 30,000 face images, only a subset of 3000 images was used due to compute and time constraints. This subset was split into 2700 training images and 300 held-out test images using a fixed random seed. The test images were not used during training and were only used for visual comparison and FID evaluation.

Before the main CelebA-HQ experiments, a butterfly image dataset was also used as an initial proof-of-concept. This stage helped the group understand how diffusion models behave on a simpler image domain. It also helped validate the notebook structure, image loading process, noising visualisation, and sample generation workflow before moving to face generation.

B. Preprocessing

For the final CelebA-HQ setup, images were resized to 128×128 . This reduced GPU memory usage and made training more practical. Each image was converted to a tensor and normalised from pixel values in $[0, 1]$ to $[-1, 1]$. Random horizontal flipping was applied to the training set as a light augmentation. The test set used deterministic preprocessing without augmentation so that evaluation remained consistent.

Separate transforms were used for training and testing. This was important because applying augmentation to the test set would change the reference distribution used for FID. Colour jitter was later tested as an augmentation strategy, but it worsened the generated colour distribution and increased FID. Therefore, the final model used only horizontal flipping.

C. Forward Diffusion

The forward diffusion process gradually adds Gaussian noise to a clean image over $T = 1000$ timesteps. At low timesteps, the image remains mostly recognisable. At high timesteps, the image approaches random noise. The model is trained to predict the added noise from the noisy image and its timestep.

The simplified training objective is:

$$L = E_{x_0, \epsilon, t} [\|\epsilon - \epsilon_\theta(x_t, t)\|^2] \quad (1)$$

where x_0 is the clean image, x_t is the noisy image at timestep t , ϵ is the true Gaussian noise, and ϵ_θ is the U-Net prediction.

D. Model Architecture

The denoising model was a U-Net implemented using the HuggingFace Diffusers library. The model accepts a noisy RGB image and timestep as input and predicts a three-channel noise residual. The architecture contains downsampling blocks, upsampling blocks, and attention blocks. The downsampling path captures increasingly abstract spatial features, while the upsampling path reconstructs image-level detail. Attention helps the model capture wider image dependencies, which is useful for maintaining global face structure.

The selected U-Net had approximately 113 million trainable parameters. This made the model expressive enough to learn complex facial structure, but it also increased memory usage and training time. This was one reason why 128×128 became the most practical final resolution.

E. Training Setup

The model was trained using mean squared error loss between predicted and true noise. AdamW was used as the optimiser. The final run used 128×128 resolution, batch size 8, a linear DDPM noise scheduler, learning rate 5×10^{-5} , and 150 training epochs. Mixed-precision training was used to reduce GPU memory usage and improve efficiency. Checkpoints were saved every 25 epochs, but the large checkpoint files were excluded from GitHub because they exceeded repository file size limits.

F. Implementation and Reproducibility

The project was implemented in Python using PyTorch, torchvision, HuggingFace Diffusers, matplotlib, and cleanfid. Most development was carried out in a Jupyter notebook so that intermediate outputs could be inspected during training. This was useful because diffusion training cannot be judged from loss alone; generated image grids and real-versus-generated comparisons were needed to understand whether the model was improving visually.

A fixed seed was used for dataset splitting and sample generation where possible. This helped keep the train-test split consistent and made repeated experiments easier to compare. However, complete reproducibility was difficult because GPU operations, random noise sampling, and the stochastic reverse diffusion process can still introduce small variations. The

final evaluation therefore treats FID as a useful comparative measure rather than an exact absolute value.

The repository was maintained using GitHub. Source code, notebooks, figures, summaries, and report files were committed, while large PyTorch checkpoint files were excluded because they exceeded GitHub file-size limits. Scratch storage was used for dataset files, generated samples, and model checkpoints because the CelebA-HQ dataset and trained checkpoints were too large for normal repository storage.

IV. EXPERIMENTS AND RESULTS

TABLE I: Summary of DDPM experiments

Experiment	Res.	Batch	Epochs	Scheduler	FID
Baseline	128	8	100	Linear	103.64
256-Cosine	256	2	30	Cosine	110.59
256-Jitter	256	4	50	Cosine	114.43
Re-run Baseline	128	8	100	Linear	105.59
Final Run	128	8	150	Linear	94.41

A. Training Loss

The training loss decreased sharply in the first few epochs and then stabilised. This shows that the model quickly learned the basic denoising objective. Later improvements were slower because the model was refining image details rather than learning the overall task from scratch. The loss curve remained somewhat noisy because each mini-batch used random timesteps and random Gaussian noise.

A key observation was that lower training loss did not always guarantee better-looking images. Diffusion training loss measures noise prediction accuracy, but generated image quality depends on how errors accumulate during the reverse sampling process. Therefore, loss curves were used together with generated samples and FID scores.

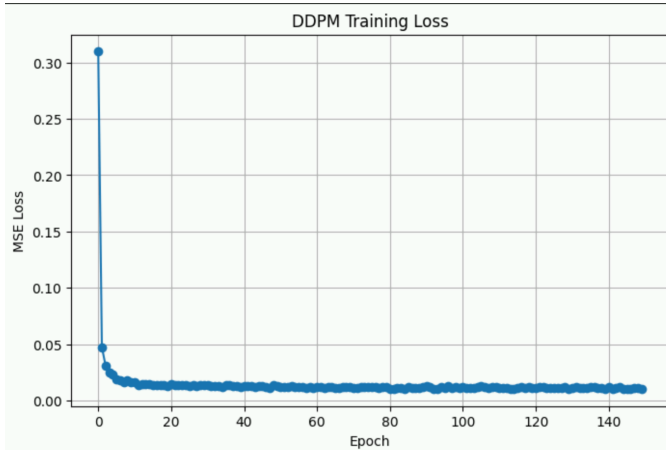


Fig. 1: Training loss curve for the DDPM experiment.

B. Butterfly Generation Trial

Before the final face generation pipeline, a butterfly diffusion experiment was used as an early trial. This helped

validate the general pipeline on a simpler dataset. The generated butterfly outputs were useful for checking whether the training and sampling process worked correctly. Although the butterfly experiment was not the main result, it contributed to the development of the final CelebA-HQ pipeline.

The butterfly experiment also helped the group understand practical issues such as image folder structure, preprocessing consistency, sample grid creation, and how generated outputs change across training. This reduced debugging time when the project moved to CelebA-HQ.



Fig. 2: Example output from the preliminary butterfly diffusion experiment.

C. Generated Face Samples

The final 128×128 model generated recognisable face-like images with visible facial structure, hair, skin regions, and pose variation. Compared with the earlier 100 epoch baseline, the 150 epoch model produced a lower FID and more coherent samples. However, the generated images were still not fully realistic. Some images contained colour shifts, blurred boundaries, distorted eyes, or unnatural lighting.

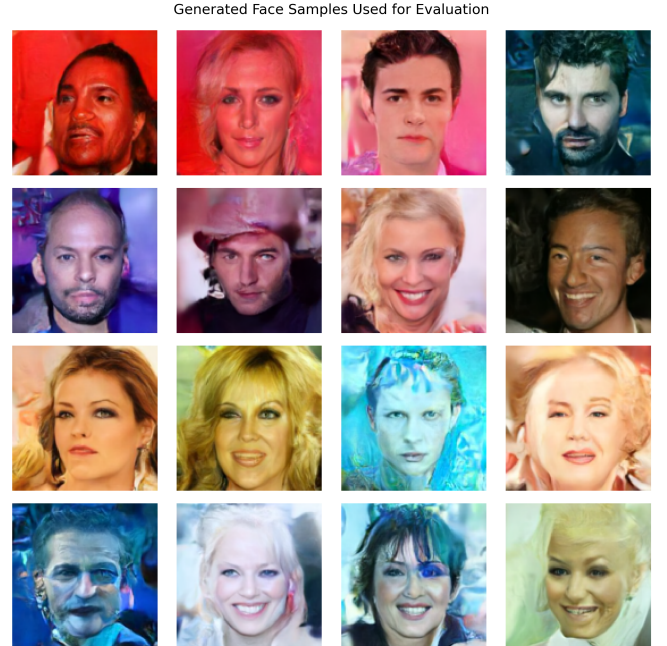


Fig. 3: Generated face samples from the final DDPM model.

D. Real vs Generated Comparison

A real-versus-generated comparison was created using held-out CelebA-HQ test images and generated samples. The real

images were sharper, more naturally coloured, and structurally more consistent. The generated samples captured the broad distribution of face images but remained less detailed. The comparison was useful because it showed both the success and limitations of the model more clearly than FID alone.

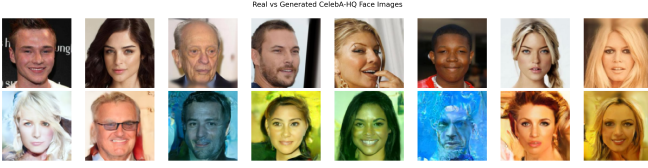


Fig. 4: Comparison between real CelebA-HQ test images and generated samples.

E. FID Evaluation

The best FID achieved was 94.41 from the final 150 epoch 128×128 run. This improved over the earlier 100 epoch baseline, which achieved 103.64. The 256×256 cosine experiment achieved 110.59, and the 256×256 colour jitter experiment achieved 114.43. These results show that simply increasing resolution did not improve generation quality under the available computational budget.

The final result suggests that for this project, longer training at a manageable resolution was more effective than moving to a larger image size. The 128×128 configuration allowed batch size 8 and more stable training, while 256×256 experiments required smaller batch sizes and trained more slowly.

Since only 300 generated images and 300 real test images were used for FID, the absolute FID values should be interpreted with caution. FID is more stable when computed over larger sample sizes, but generating thousands of images was expensive because DDPM sampling requires many reverse diffusion steps. Therefore, FID was mainly used as a relative comparison between configurations.

F. Experimental Challenges

Several practical challenges were encountered during the project. The first major issue was storage. CelebA-HQ, generated images, and model checkpoints required much more space than normal project files. The trained checkpoint for the final model exceeded GitHub upload limits, so model files were excluded from version control and only the notebook, output figures, and summaries were committed.

A second challenge was compute time. Training at 256×256 resolution required smaller batch sizes and was slower per epoch. Sampling was also expensive because each generated image required hundreds of denoising steps. This limited how many experiments could be performed and made it difficult to evaluate very large sample sets.

A third challenge was interpretation. The training loss could decrease even when visual artefacts remained. This made it necessary to inspect generated grids, compare real and generated images, and compute FID. The group therefore used both qualitative and quantitative evidence when selecting the final model.

V. DISCUSSION

The experiments show that the best trade-off between visual quality, FID, and training time was achieved at 128×128 resolution. Although higher-resolution generation is usually desirable, the 256×256 experiments were more difficult to train from scratch. They required more GPU memory, smaller batch sizes, and longer sampling time. With only 2700 training images and limited compute, the model struggled to learn stable high-resolution face generation.

The improvement from FID 103.64 to 94.41 suggests that extending the 128×128 training run was beneficial. The final model had more time to refine the denoising process, and the lower learning rate helped stabilise training. This indicates that the earlier 100 epoch model was likely still undertrained rather than overtrained. If the model were strongly overfitting, the generated images would be expected to closely resemble training examples or lose diversity. Instead, the outputs remained varied but imperfect, suggesting that the model still needed more training data, compute, or architectural improvement.

The loss curve alone was not enough to judge model quality. The MSE loss measures how well the network predicts Gaussian noise, but it does not directly measure whether the final generated image looks realistic to a human observer. A model can achieve a lower denoising loss while still producing samples with colour artefacts or distorted facial features. Therefore, FID and visual inspection were both necessary.

The colour jitter experiment was useful as an ablation, even though it did not improve results. In classification tasks, colour jitter often improves robustness. In this generative task, however, the model needed to learn the real image distribution as closely as possible. Colour jitter changed the training image distribution and likely encouraged stronger colour variation in generated outputs. This explains why the jitter experiment produced worse colour artefacts and a higher FID.

The 256×256 experiments also showed that increasing image size alone does not guarantee better generation. Higher resolution increases the number of pixels the model must predict and makes the reverse diffusion process more difficult. Since the batch size had to be reduced at 256×256 , the optimisation became less stable. The 128×128 setup allowed batch size 8 and longer training, which was more suitable for the available resources.

VI. CONCLUSION AND FUTURE WORK

This project successfully implemented a DDPM pipeline for image generation, beginning with a butterfly dataset trial and progressing to CelebA-HQ face generation. The final system includes dataset loading, preprocessing, forward diffusion visualisation, U-Net denoising, training, sample generation, real-versus-generated comparison, and FID evaluation. The project therefore covers the complete workflow from raw image data to quantitative generative model evaluation.

The strongest experiment achieved an FID score of 94.41 using 128×128 images, batch size 8, a linear scheduler, learning rate 5×10^{-5} , and 150 training epochs. This was the best result among all tested configurations. The result

shows that longer training at a manageable resolution was more effective than increasing resolution too early. The final generated images contain recognisable face structure, but they still show limitations such as colour shifts, blur, and occasional facial distortion.

Future work should include training with a larger subset of CelebA-HQ, continuing from saved checkpoints, evaluating more inference steps, testing DDIM sampling, and using additional quality metrics beyond FID. A larger training set would likely improve diversity and realism, while DDIM sampling could reduce generation time. Latent diffusion would also be a promising direction because it performs diffusion in a compressed latent space rather than directly in pixel space, making higher-resolution generation more practical.

The project could also be improved by separating training, generation, and evaluation into standalone scripts. This would make experiments easier to reproduce without rerunning the full notebook. A more complete evaluation could include FID across multiple random seeds, side-by-side human visual inspection, and additional metrics such as KID or precision-recall.

REFERENCES

- [1] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 6840–6851.
- [2] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, “Score-based generative modeling through stochastic differential equations,” in *International Conference on Learning Representations*, 2021.
- [3] P. Dhariwal and A. Nichol, “Diffusion models beat GANs on image synthesis,” in *Advances in Neural Information Processing Systems*, vol. 34, 2021, pp. 8780–8794.
- [4] A. Q. Nichol and P. Dhariwal, “Improved denoising diffusion probabilistic models,” in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 139. PMLR, 2021, pp. 8162–8171.
- [5] T. Karras, T. Aila, S. Laine, and J. Lehtinen, “Progressive growing of GANs for improved quality, stability, and variation,” in *International Conference on Learning Representations*, 2018.
- [6] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional networks for biomedical image segmentation,” in *Medical Image Computing and Computer-Assisted Intervention*. Springer, 2015, pp. 234–241.
- [7] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations*, 2019.
- [8] P. von Platen, S. Patil, A. Lozhkov, P. Cuenca, N. Lambert, K. Rasul, M. Dompé, and T. Wolf, “Diffusers: State-of-the-art diffusion models,” 2022. [Online]. Available: <https://github.com/huggingface/diffusers>
- [9] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, “GANs trained by a two time-scale update rule converge to a local Nash equilibrium,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 6627–6638.
- [10] J. Song, C. Meng, and S. Ermon, “Denoising diffusion implicit models,” in *International Conference on Learning Representations*, 2021.
- [11] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-resolution image synthesis with latent diffusion models,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 10 684–10 695.

APPENDIX A

AUTHOR CONTRIBUTIONS

Johan Joe Paul – 6946540:

Johan contributed to the overall project setup, repository management, and coordination of the final project structure. Responsibilities included organising the GitHub repository,

maintaining notebook structure, managing commits, handling repository cleanup, and resolving version-control issues involving large model checkpoints and generated result files. Johan also helped structure the final report, integrate generated figures into the document, and improve the overall readability and consistency of the written content.

Additional contributions included dataset setup verification, checking preprocessing consistency, validating saved outputs, and helping maintain reproducibility across experiments. Johan also worked on final README improvements, repository documentation, and preparation of the final GitHub submission. One challenge encountered during this stage was handling GitHub storage limitations for large diffusion checkpoints and generated outputs. This required careful use of `.gitignore`, scratch storage, and repository restructuring to ensure that only the required files were committed while preserving reproducibility.

Juby Kumpilunilkunnathil Jose – 6944166:

Juby contributed primarily to DDPM model development, U-Net configuration, scheduler experimentation, and model validation. Work included configuring the HuggingFace Diffusers U-Net architecture, selecting scheduler types, testing cosine and linear schedulers, and analysing how different hyperparameters affected generation quality. Juby also contributed to experiments involving image resolution changes, learning-rate adjustments, batch-size modifications, and colour jitter augmentation.

Additional work included analysing the effect of higher-resolution 256×256 training and comparing it against the more stable 128×128 configuration. Juby helped investigate why larger image sizes did not improve FID under the available training budget and supported decisions regarding the final selected configuration. One challenge during this process was balancing image quality improvements against GPU memory constraints, especially when experimenting with higher resolutions and reduced batch sizes.

Shan Syrus – 6946315:

Shan contributed mainly to image generation, output evaluation, result visualisation, and comparison analysis. Responsibilities included generating image grids, saving generated samples, organising real-versus-generated comparisons, computing FID scores, and preparing evaluation summaries for inclusion in the report. Shan also helped analyse the visual quality of generated outputs and compare differences across multiple experiments.

Additional contributions included preparing evaluation folders for clean-fid computation, verifying generated sample consistency, and assisting with the interpretation of experimental results. Shan also supported the selection of representative generated outputs for the final report figures. One challenge encountered during this stage was that visual quality and FID did not always improve together, requiring careful comparison between quantitative scores and qualitative inspection of generated samples.

Kezia Mariam Mathew – 6943290:

Kezia contributed to preprocessing, train-test splitting, DataLoader setup, scratch environment configuration, repeated training experiments, and execution of the final improved 150 epoch run. Responsibilities included configuring dataset transforms, validating training and test preprocessing consistency, constructing PyTorch DataLoaders, and ensuring that training batches were loaded efficiently during diffusion training.

Kezia also played a major role in managing scratch storage and ensuring that datasets, generated images, and checkpoints could be processed despite storage limitations. Additional work included rerunning baseline experiments, validating training stability, and supporting the final 150 epoch configuration that achieved the best FID score of 94.41. One major challenge during this stage involved managing long training durations and ensuring that generated outputs and evaluation files were saved correctly without corruption or storage conflicts.

Dhanya Vinod Kumar – 6961891:

Dhanya contributed to the training pipeline, optimisation setup, checkpoint handling, and stability of the DDPM training process. Work included configuring the AdamW optimiser, implementing the denoising loss setup, managing checkpoint saving, and supporting mixed-precision training for reduced GPU memory usage. Dhanya also contributed to debugging training instability and ensuring that model checkpoints could be resumed correctly during longer experiments.

Additional responsibilities included monitoring training behaviour, helping analyse loss curves, and supporting adjustments to training duration and optimisation settings. Dhanya also assisted with debugging issues related to noise prediction behaviour and helped ensure that the reverse diffusion sampling process produced usable generated outputs. One challenge encountered during training was that decreasing loss values did not always correspond to visually improved outputs, making it necessary to rely on both qualitative inspection and FID evaluation when comparing models.

APPENDIX B EXPERIMENTAL NOTES

The final report uses the 128×128 , batch size 8, 150 epoch configuration as the primary result because it achieved the best balance between visual quality, training stability, and quantitative evaluation. Although higher-resolution experiments were initially expected to improve realism, the 256×256 runs required significantly more GPU memory, reduced the achievable batch size, and increased training instability. Under the available compute budget, the lower-resolution configuration ultimately performed better.

The butterfly dataset experiment was included as an initial proof-of-concept before moving to CelebA-HQ. This stage helped validate the overall diffusion workflow, including dataset loading, preprocessing, noising visualisation, reverse denoising, training behaviour, and sample generation. Although the butterfly results were not the primary contribution of the project, they played an important role in understanding

the behaviour of diffusion models before scaling to more complex face-generation experiments.

Throughout the project, multiple practical limitations affected experimentation. Diffusion models required long training times, especially during image sampling, where hundreds of denoising steps were needed for each generated image. Storage constraints also became significant because generated outputs, checkpoints, and datasets occupied substantial disk space. As a result, scratch storage was heavily used during development while GitHub was limited to notebooks, figures, summaries, scripts, and documentation.

The project also highlighted the importance of combining qualitative and quantitative evaluation. Training loss alone was not sufficient for model selection because lower denoising loss did not always produce visually better outputs. Real-versus-generated comparison figures and FID evaluation were therefore used together to identify the strongest final model.

Finally, the repository was organised to support reproducibility and future extension. The GitHub repository includes notebooks, setup scripts, generated result figures, evaluation summaries, and documentation so that future experiments can continue from the implemented baseline. Potential future extensions include DDIM sampling, latent diffusion, larger training subsets, improved evaluation metrics, and standalone training or evaluation scripts outside the notebook workflow.